LangGraph Debugging — Worked Example: Solution

> Back to the exercise · Round 1: Basic · Round 2: Intermediate · Round 3: Real-world · Round 4: Multi-agent · Hints

Story — `langgraph_debugging_practice.py`: graph bugs tend to live in the wiring itself — route key names, how state fields get merged, when exactly a loop is supposed to stop — not inside any single node's own code. Each fix here comes with a test that either runs a tiny real graph, or checks the routing function directly, using stand-in nodes and no model call at all. **If not:** you'd only ever see these bugs during a full, slow, paid run, and end up blaming the model for what's really a wiring mistake.

Every fix and test below goes in `practice/langgraph_debugging_practice.py`, and runs with `pytest langgraph_debugging_practice.py -v` from inside `practice/`. None of the tests need an API key.

- Round 1: Basic
- Round 2: Intermediate
- Round 3: Real-world
- Round 4: Multi-agent
- Real cause vs. symptom fix

Round: Basic

Real cause: `route()` returns `"finish"`, but the mapping given to `add_conditional_edges()` only has the keys `"call_tool"` and `"done"`. LangGraph looks the returned word up in the mapping, doesn't find it, and raises `KeyError: 'finish'` — a plain naming mismatch between the routing function and its mapping. This is Doc09's own Edge cases exercise (`unhandled_routing_value`).

Story: a bare `KeyError: 'finish'` gives you almost nothing to go on by itself, so this round trains the move of searching your own code for that exact string, to find who produces it and who's actually supposed to accept it in the routing map. **If not:** you'd go digging inside LangGraph's own source for a bug that's really a one-word mismatch sitting in your own file.

The fix:

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class AgentState(TypedDict):
    needs_tool: bool

def agent_node(state):
    return {}                        # stand-in for the real agent

def tool_node(state):
    return {"needs_tool": False}     # stand-in: the tool ran, done now

def route(state):
    if state["needs_tool"]:
        return "call_tool"
    # why: must be a key in the mapping below — "done", not "finish"
    return "done"

def build_graph():
    graph = StateGraph(AgentState)
    graph.add_node("agent", agent_node)
    graph.add_node("tool_node", tool_node)
    graph.add_edge(START, "agent")
    graph.add_conditional_edges("agent", route, {
        "call_tool": "tool_node",
        "done": END,
    })
    graph.add_edge("tool_node", "agent")
    return graph.compile()
```

Test that would have caught it:

```
def test_graph_can_take_the_done_branch():
    result = build_graph().invoke({"needs_tool": False})
    assert result["needs_tool"] is False

def test_graph_can_take_the_tool_branch_and_finish():
    result = build_graph().invoke({"needs_tool": True})
    assert result["needs_tool"] is False
```

One test per branch: a suite that only ever tested the "keep going" branch would never reach the one with the typo.

Round: Intermediate

Real cause: for a plain state field (no reducer), each node's update simply **replaces** the old value. `calculate_node`'s `{"findings": {"calc": 42}}` doesn't merge with `search_node`'s earlier `{"findings": {"search": ...}}` — it overwrites the whole dict, because nothing told LangGraph how to combine two updates to the same field.

Story: the log makes it look like "the model forgot" — but the state itself actually lost the data before the model ever had a chance to see it. This round trains inspecting the state at each step of the graph before blaming the model's reasoning for something that was never in front of it. **If not:** you'd add "remember earlier findings" to the prompt, and absolutely nothing would change, because the data genuinely isn't there.

The fix:

```
from typing import Annotated, TypedDict
from langgraph.graph import StateGraph, START, END

def merge_findings(current, update):
    # why: a reducer tells LangGraph HOW to combine old and new —
    # here, keep every key from both dicts
    merged = dict(current)
    for key in update:
        merged[key] = update[key]
    return merged

class FindingsState(TypedDict):
    task: str
    findings: Annotated[dict, merge_findings]

def search_node(state):
    # stand-in for run_search(state["task"])
    return {"findings": {"search": {"refund_policy": "30 days"}}}

def calculate_node(state):
    # stand-in for run_calculation(state["task"])
    return {"findings": {"calc": 42}}

def build_findings_graph():
    graph = StateGraph(FindingsState)
    graph.add_node("search", search_node)
    graph.add_node("calculate", calculate_node)
    graph.add_edge(START, "search")
    graph.add_edge("search", "calculate")
    graph.add_edge("calculate", END)
    return graph.compile()
```

Test that would have caught it:

```
def test_findings_from_both_nodes_are_kept():
    result = build_findings_graph().invoke({"task": "x", "findings": {}})
    assert "search" in result["findings"]
    assert "calc" in result["findings"]
```

The test has to run both nodes through a real graph — only the graph applies the reducer. Calling the two node functions by hand and merging with `dict.update()` would skip the reducer and prove nothing.

Round: Real-world

Real cause: `route()` only checks confidence against a fixed threshold — there's no counter of how many search rounds have happened. Most questions pass 0.5 within a couple of rounds. A certain kind of ambiguous question stays "somewhat relevant, never enough" forever, and with nothing else to stop it, the loop runs to LangGraph's hard recursion limit — the *symptom* (a recursion error) fires 25 steps after the *real cause* (a loop with only one way out) let it start.

Story: `GraphRecursionError` shows no stack frame from your own node code, and that absence is the real clue: nothing actually crashed, the *routing itself* simply never let the graph exit. This round trains giving every loop two separate exits — "good enough to stop" and "tried enough times to give up" — not just one. **If not:** you'd just raise the recursion limit, and the exact same questions would simply loop longer and cost more before failing the same way.

The fix:

```
from typing import TypedDict

class SearchState(TypedDict):
    task: str
    confidence: float
    search_rounds: int

def route_search(state):
    if state["confidence"] >= 0.5:
        return "done"
    # why: the second way out — stop after 4 tries, confident or not
    if state["search_rounds"] >= 4:
        return "give_up"
    return "search_again"
```

Each search node adds 1 to `search_rounds` in its update, and `"give_up"` routes to a node that returns a clear "couldn't find a confident answer" result.

Test that would have caught it:

```
def test_search_loop_gives_up_after_four_rounds():
    state = {"task": "x", "confidence": 0.3, "search_rounds": 4}
    assert route_search(state) == "give_up"

def test_search_loop_keeps_going_before_the_limit():
    state = {"task": "x", "confidence": 0.3, "search_rounds": 1}
    assert route_search(state) == "search_again"
```

The routing function is plain Python, so the "never confident" case can be tested with a hand-made state in milliseconds — no need to run 25 real search rounds.

Round: Multi-agent

Real cause: two bugs stacked on top of each other. First, `review_route` has the same problem as Round 3 — no cap on revision rounds. Second, the Reviewer keeps rejecting because the "compliance detail" it wants was never written into `research_findings` at all (a Research-agent gap), so the Writer is asked, every round, for something it can't produce. Neither shows up testing the Writer/Reviewer pair with hand-written examples, because those always include findings the Writer *can* satisfy.

Story: the loop here actually has two distinct causes, and each fix addresses a different one — a hard cap stops the endless loop regardless of *why* it's looping, while looking one layer back uncovers the actual reason it loops at all. This round trains fixing both causes, not stopping at the first one you happen to notice. **If not:** the cap alone would just turn "loops forever" into "gives up every single time," and the real missing compliance detail would stay missing, unaddressed.

The fix:

```
from typing import TypedDict

class ReviewState(TypedDict):
    task: str
    research_findings: str
    draft: str
    review_passed: bool
    revision_count: int

def count_revision(state):
    # how: the Writer node adds this to its update on every draft
    return {"revision_count": state["revision_count"] + 1}

def review_route(state):
```

```
    if state["review_passed"]:
        return "done"
    # why: Doc11's Build Task rule — a hard limit on revision rounds
    if state["revision_count"] >= 3:
        return "give_up"
    return "revise"
```

The graph maps `"give_up"` to a `give_up_report` node that says clearly the agents couldn't agree. Then go one layer back: make the Reviewer's rejection name what's missing from `research_findings`, so the gap in the Research agent's output is visible instead of hidden inside an endless loop.

Test that would have caught it:

```
def test_review_loop_gives_up_after_three_revisions():
    state = {"task": "x", "research_findings": "", "draft": "d",
             "review_passed": False, "revision_count": 3}
    assert review_route(state) == "give_up"

def test_each_draft_counts_as_one_revision():
    state = {"task": "x", "research_findings": "", "draft": "d",
             "review_passed": False, "revision_count": 1}
    assert count_revision(state) == {"revision_count": 2}
```

The first test proves the loop can end; the second proves the counter really moves, so the limit is actually reached.

Real cause vs. symptom fix

A symptom fix here would be wrapping `invoke()` in a `try/except KeyError` (Round 1), telling the model in the prompt to "remember all earlier findings" (Round 2), or raising `recursion_limit` to 100 (Rounds 3 and 4). Each makes one visible failure go away while the wiring stays wrong: the mapping still doesn't match, state still overwrites itself, and loops still have only one exit — so the next unusual input hits the same wall, later and more expensively. The real fixes all change the graph's structure — matching route names, a reducer that says how to merge, and a second exit on every loop — which is why they hold for inputs you haven't tried yet.